```
consumer_secret =  "consumer_secret"
kafka_endpoint = "kafkabroker_host:9092"
kafka_topic = "kafka_topic"
twitter_hash_tag = "hadoop"
time_limit = 60

class StdOutListener(StreamListener):
    def __init__(self, time_limit=time_limit):
        self.start_time = time.time()
        self.limit = time_limit
        super(StdOutListener, self).__init__()
    def on_data(self, data):
        if (time.time() - self.start_time) < self.limit:
            producer.send_messages(kafka_topic, data.
encode('utf-8'))
            print (data)
            return True
        exit(0)
    def on_error(self, status):
        print (status)
#connect to kafka broker server
kafka = KafkaClient(kafka_endpoint)
#create a producer object and publish message from twitter
source.
producer = SimpleProducer(kafka)
l = StdOutListener()
auth = OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_token_secret)
stream = Stream(auth, l)
stream.filter(track=twitter_hash_tag)
```

The code for *WordCount.py* is:

```
#!/usr/bin/python
from __future__ import print_function
import os
import findspark
findspark.init('/opt/cloudera/parcels/SPARK2/lib/spark2')
import sys
from pyspark import SparkContext
from pyspark.streaming import StreamingContext
from pyspark.streaming.kafka import KafkaUtils

zkQuorum = 'zookeeperserver_host:2181'
topic = 'kafka_topic'
if __name__ == "__main__":
    #create spark context
    sc = SparkContext(appName="PythonStreamingKafkaWordCount")
    #create streaming spark context object with streaming
interval of 10sec
    ssc = StreamingContext(sc, 10)
    #read the streaming data from a kafka topic
    kvs = KafkaUtils.createStream(ssc, zkQuorum, "spark-
streaming-consumer", {topic: 1})
    lines = kvs.map(lambda x: x[1])
    counts = lines.flatMap(lambda line: line.split(" ")).
map(lambda word: (word, 1)).reduceByKey(lambda a, b: a+b)
    counts.pprint()
    ssc.start()
    ssc.awaitTermination()
```

How to debug Spark applications

Most Spark users follow their applications' progress via the Spark Web UI, driver logs and executor logs. Spark History Server is the Web UI for completed and running (incomplete) Spark applications. It is an extension of Spark's Web UI. It displays tables with information about jobs, stages, tasks, executors, RDDs and more. These are described below.

Job - A parallel computation consisting of multiple tasks that gets spawned in response to a Spark action.

Stage - Each job gets divided into smaller sets of tasks called stages that depend on each other (similar to the Map and Reduce stages in MapReduce).

Task - A unit of work that will be sent to one executor.

Executor - A process launched for an application on a worker node, which runs tasks and keeps data in memory or disk storage. Each application has its own executors.

RDD - Resilient Distributed Dataset (RDD), which is a fault-tolerant collection of elements that can be operated on in parallel. There are two ways to create RDDs—an existing collection in your driver program, or referencing a data set in an external storage system, such as a shared file system, HDFS, HBase or any data source offering a Hadoop InputFormat.

If you wish to create a Spark environment on top of a Cloudera distribution of Hadoop (CDH) cluster, with YARN as the cluster manager, and to scale your cluster seamlessly for learning and R&D purposes, do visit my GitHub repo *https://github.com/rkkrishnaa/vajra*.

Spark is adopted by many companies all over the world. It attracts data scientists and data analysts with its rich set of machine learning libraries and very fast data processing capabilities. Many research activities are going on to integrate Spark with popular cluster managers like Kubernetes. Popular cloud providers such as AWS, Azure and GCP provide Kubernetes cluster service. This will accelerate the provisioning and management of the Spark cluster. END

References

[1] *https://spark.apache.org/docs/2.1.1/index.html*
[2] *https://spark.apache.org/docs/2.1.1/streaming-programming-guide.html*
[3] *https://www.cloudera.com/documentation/enterprise/5-11-x/PDF/cloudera-spark.pdf*

By: Radhakrishnan Ramakrishnan

The author is a DevOps engineer with Dattatreya Consultancy Services Pvt Ltd, Madurai. He has expertise in Linux, OpenStack, AWS, Web development and Hadoop administration.